



University of Idaho

Department of Computer Science

CS 4727/5727

Adversarial Machine Learning

Dr. Alex Vakanski



Lecture 8

Defenses against Evasion Attacks



Lecture Content

8.1 Adversarial ML Defense Methods

8.1.1 Model Robustness

8.2 Adversarial Examples Detection

8.2.1 Auxiliary Detection Model, MagNet Defense, Statistical Detection Methods, Prediction Consistency Methods

8.3 Gradient Obfuscation

8.3.1 Defensive Distillation

8.4 Robust Optimization

8.4.1 Adversarial Training

8.4.2 Regularization Methods

8.4.3 Certified Defenses

8.4.3.1 Cohen (2019) Certified Adversarial Robustness via Randomized Smoothing

8.4.3.2 Carlini (2022) (Certified!!) Adversarial Robustness for Free!

AML Defense Categories

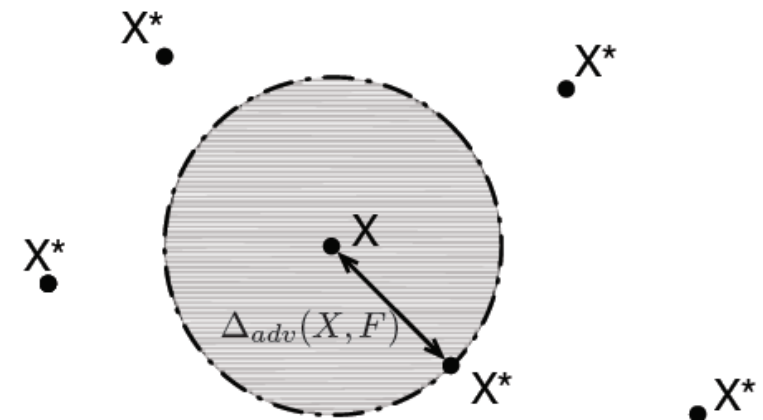
8.1 Adversarial ML Defense Methods

- The defense strategies against adversarial attacks can be categorized into three main groups:
 - *Adversarial examples detection*
 - Design a method for distinguishing clean from adversarial examples
 - E.g., train a binary classification model to detect adversarial examples
 - *Gradient obfuscation*
 - Design a method to hide the gradients in the target ML model
 - To make attacks that use the gradients ineffective
 - *Robust optimization*
 - Design a method to increase the robustness of the target ML model to adversarial examples
 - E.g., apply adversarial training by using both clean and attacked examples

Model Robustness

8.1.1 Model Robustness

- Before covering defense strategies, let's first study an approach for measuring the robustness of an ML model
- A **robust ML model** is less sensitive to adversarial perturbations to the inputs
 - A robust model should output consistent predictions for all inputs in the neighborhood of an input sample
 - The notion of **neighborhood** can be defined by a norm (e.g., an ℓ_p norm)
- The concept of robustness of a classification model F in the neighborhood of an input X is illustrated in the figure
 - The larger the **neighborhood** $\Delta_{adv}(X, F)$ around the input X is (the shaded circle in the figure), the more robust the model F is
 - The prediction by the model F for all samples located inside the shaded circle will be correct (classified as the true-class label)
 - Outside the shaded circle, the model F will classify all samples X^* differently than X



Model Robustness

8.1.1 Model Robustness

- For an input distribution P , the **robustness** of a trained ML model F , $\rho_{\text{adv}}(F)$, is measured as the expected value of the neighborhood $\Delta_{\text{adv}}(X, F)$

$$\rho_{\text{adv}}(F) = \mathbb{E}_{X \sim P}[\Delta_{\text{adv}}(X, F)]$$

- The term $\Delta_{\text{adv}}(X, F)$ is related to the **minimal perturbation** δ required to misclassify the sample X **in each of the other classes**

$$\Delta_{\text{adv}}(X, F) = \min\{\|\delta\|: F(X + \delta) \neq F(X)\}$$

- I.e., the proposed measure of robustness of the model F for the sample X is the norm of the minimal perturbation δ
- The higher the norm of the minimal perturbation required to misclassify all input samples, the more robust the model is to adversarial samples**
- For a trained ML model $F(X)$, the robustness is estimated by the average value of the minimal perturbation for a test dataset of M samples, i.e.,

$$\rho_{\text{adv}}(F) \approx \frac{1}{M} \sum_{i=1}^M \min\|\delta_i\| = \frac{1}{M} \sum_{i=1}^M \Delta_{\text{adv}}(X_i, F)$$

Adversarial Examples Detection

8.2 Adversarial Examples Detection

- *Adversarial examples detection* methods are designed to distinguish adversarial examples from regular clean examples
 - If the defense method detects that an input example is adversarial, it will refuse to send the example to the target ML classifier for predicting its class label
- Adversarial examples detection defense methods can be further divided into:
 - Auxiliary detection model
 - Statistical methods
 - Prediction consistency methods
- Limitation of the defense strategies based on adversarial examples detection is that they can be less effective in identifying examples created using unknown adversarial attacks

Auxiliary Detection Model

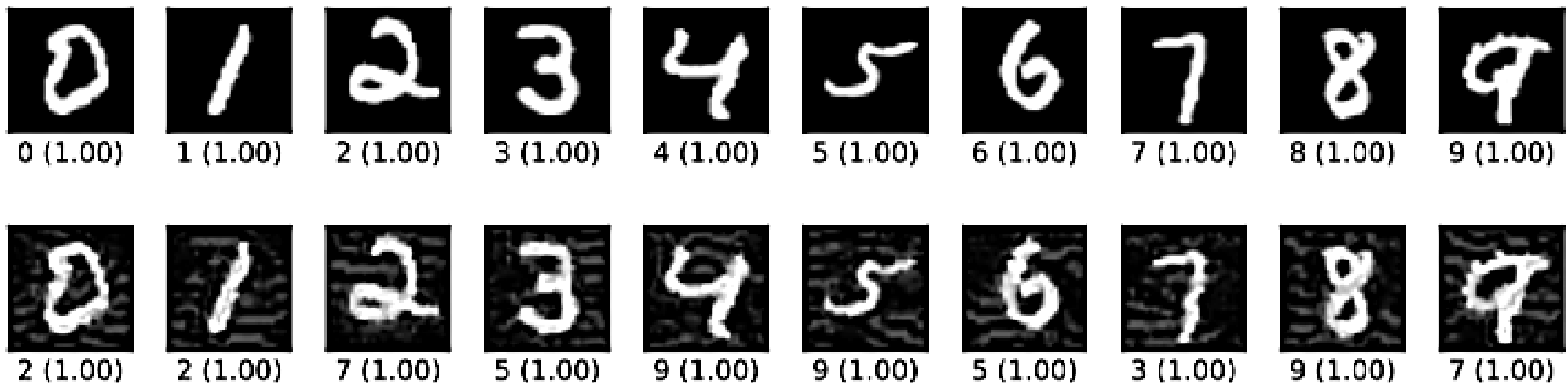
8.2 Adversarial Examples Detection

- An *auxiliary detection model* D is trained on regular and adversarial examples to perform a binary classification
 - Adversarial examples are typically created using different attack methods, to increase the effectiveness of the detection model
 - If an input example is classified as benign, then it is safe to be fed to the target classifier F to predict its class
- The auxiliary detection model D is required to have a high accuracy in correctly classifying both adversarial and clean examples

Auxiliary Detection Model

8.2 Adversarial Examples Detection

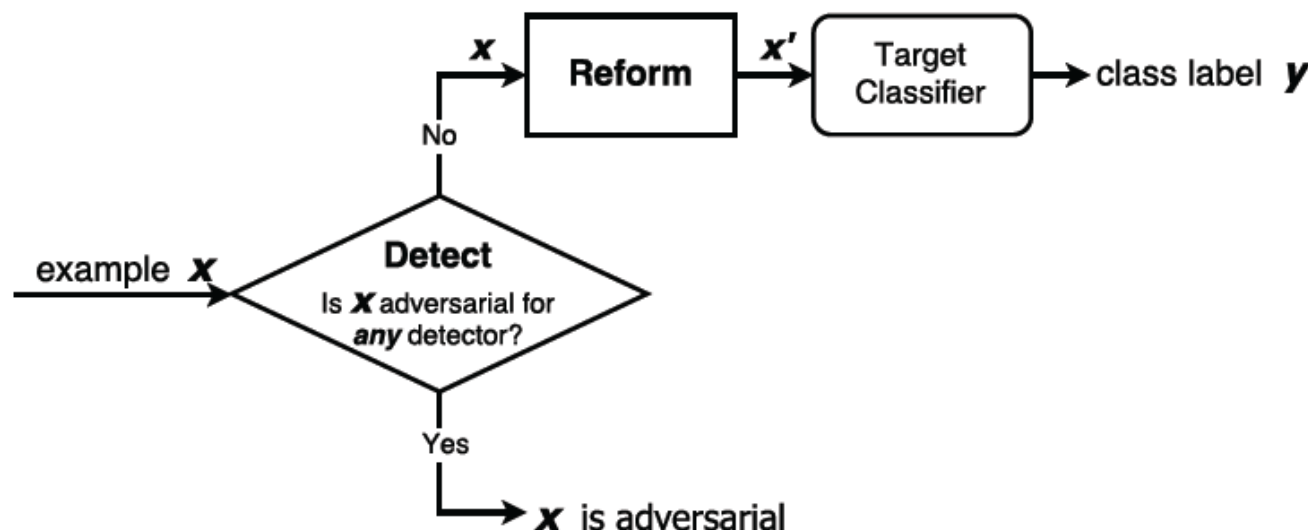
- [Gong \(2017\) Adversarial and Clean Data Are Not Twins](#)
- Train an auxiliary NN as a **binary classifier** to distinguish adversarial images and clean images
 - Figure: upper row (clean images), bottom row (adversarial FGSM) images
 - Predicted image labels and probability (in parenthesis) are shown for each image
 - The auxiliary NN achieved over 99% accuracy in binary classification, i.e., correctly predicting both clean and adversarial images



MagNet Defense

8.2 Adversarial Examples Detection

- [Meng \(2017\) MagNet: a Two-Pronged Defense against Adversarial Examples](#)
- *MagNet defense* approach uses two networks:
 - **Detector** network – an autoencoder to detect adversarial examples based on exceeding a threshold for the reconstructed outputs
 - **Reformer** network – a denoising autoencoder to remove adversarial perturbations
- The detector model is used for identifying samples with large perturbations
 - The reformer model is used for removing small perturbations in images



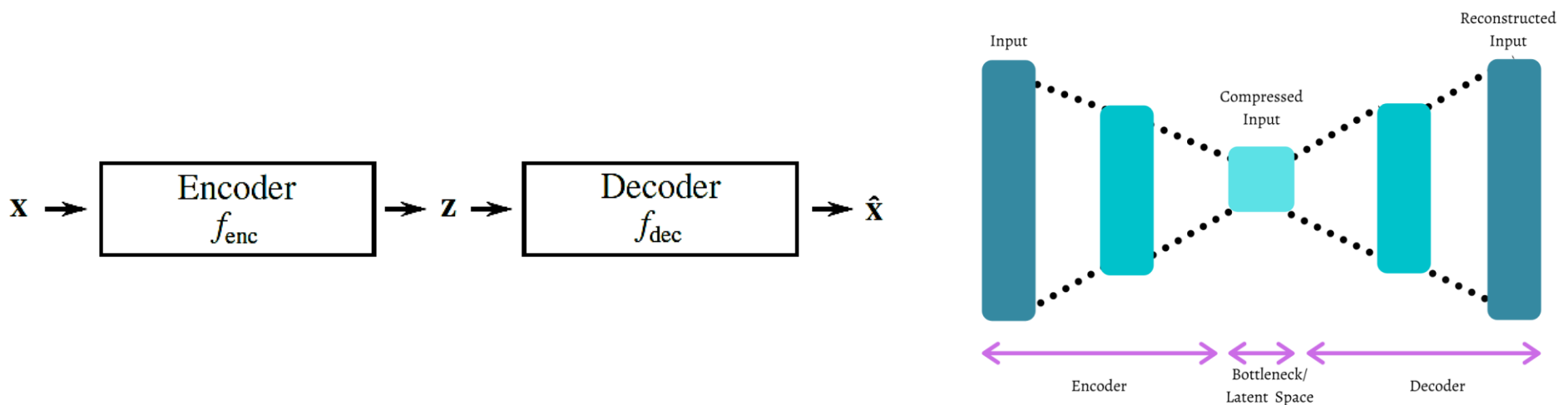
MagNet Defense

8.2 Adversarial Examples Detection

- The *detector* in MagNet is an autoencoder NN
 - It is trained **only on clean examples**
 - At test time, inputs that have a large reconstruction error are rejected as adversarial
- The *reconstruction error* for an input sample is

$$E(\mathbf{x}) = \|\mathbf{x} - \hat{\mathbf{x}}\|_2 = \|\mathbf{x} - f_{\text{dec}}(f_{\text{enc}}(\mathbf{x}))\|_2$$

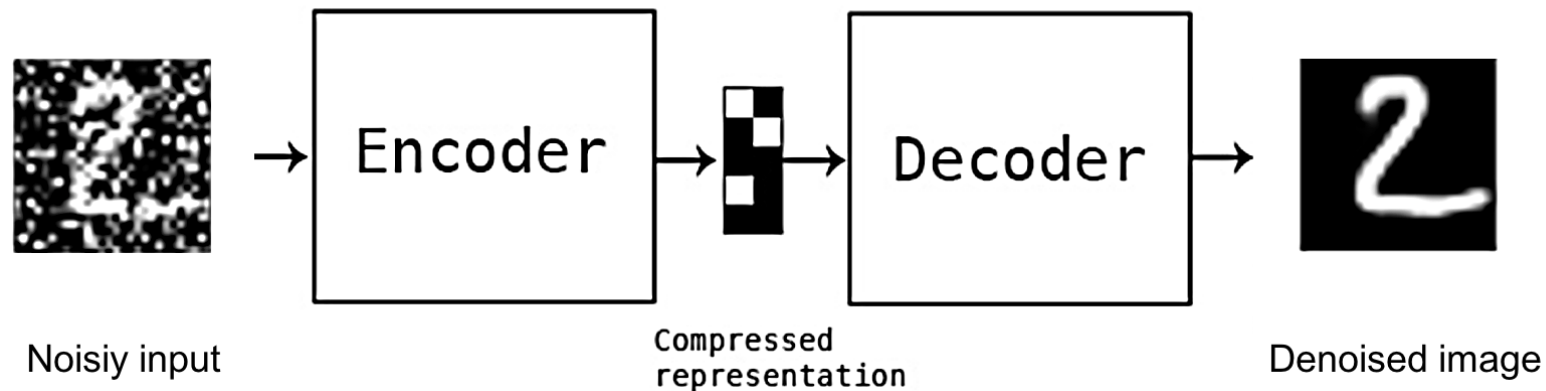
- Since the model is trained only on clean samples, the reconstruction error for unseen clean samples will be small



MagNet Defense

8.2 Adversarial Examples Detection

- The *reformer* is also a denoising autoencoder network, that is trained to clean adversarial images of adversarial perturbations
- The reformer is trained on clean samples with added Gaussian noise
 - This results in AE reconstructions of adversarial samples that are purified of the perturbations, and hence, are close to the distribution of clean samples



MagNet Defense

8.2 Adversarial Examples Detection

- Results from the implementation against black-box attacks
- MagNet effectively defends against FGSM, Iterative FGSM, DeepFool, and C&W attacks on MNIST and CIFAR-10
 - It is especially effective against C&W, as the best attack at the time of publication: e.g., the accuracy with no defense of 0% increased to 99% with defense on MNIST

(a) MNIST

Attack	Norm	Parameter	No Defense	With Defense
FGSM	L^∞	$\epsilon = 0.005$	96.8%	100.0%
FGSM	L^∞	$\epsilon = 0.010$	91.1%	100.0%
Iterative	L^∞	$\epsilon = 0.005$	95.2%	100.0%
Iterative	L^∞	$\epsilon = 0.010$	72.0%	100.0%
Iterative	L^2	$\epsilon = 0.5$	86.7%	99.2%
Iterative	L^2	$\epsilon = 1.0$	76.6%	100.0%
Deepfool	L^∞		19.1%	99.4%
Carlini	L^2		0.0%	99.5%
Carlini	L^∞		0.0%	99.8%
Carlini	L^0		0.0%	92.0%

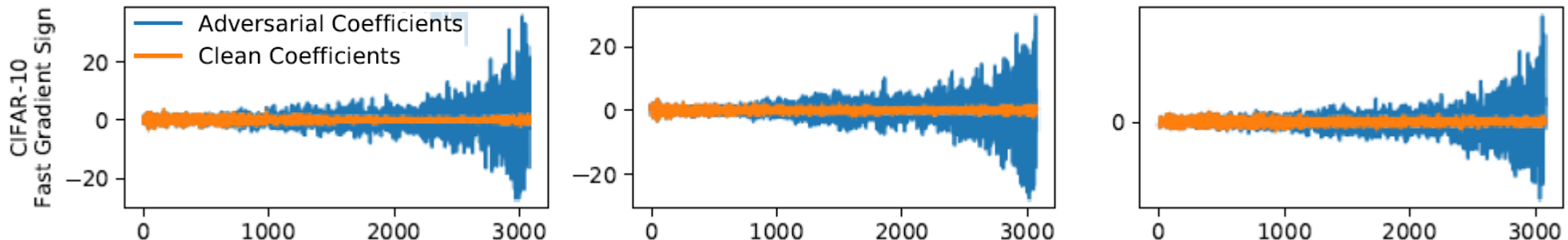
(b) CIFAR

Attack	Norm	Parameter	No Defense	With Defense
FGSM	L^∞	$\epsilon = 0.025$	46.0%	99.9%
FGSM	L^∞	$\epsilon = 0.050$	40.5%	100.0%
Iterative	L^∞	$\epsilon = 0.010$	28.6%	96.0%
Iterative	L^∞	$\epsilon = 0.025$	11.1%	99.9%
Iterative	L^2	$\epsilon = 0.25$	18.4%	76.3%
Iterative	L^2	$\epsilon = 0.50$	6.6%	83.3%
Deepfool	L^∞		4.5%	93.4%
Carlini	L^2		0.0%	93.7%
Carlini	L^∞		0.0%	83.0%
Carlini	L^0		0.0%	77.5%

Statistical Detection Methods

8.2 Adversarial Examples Detection: Statistical Detection Methods

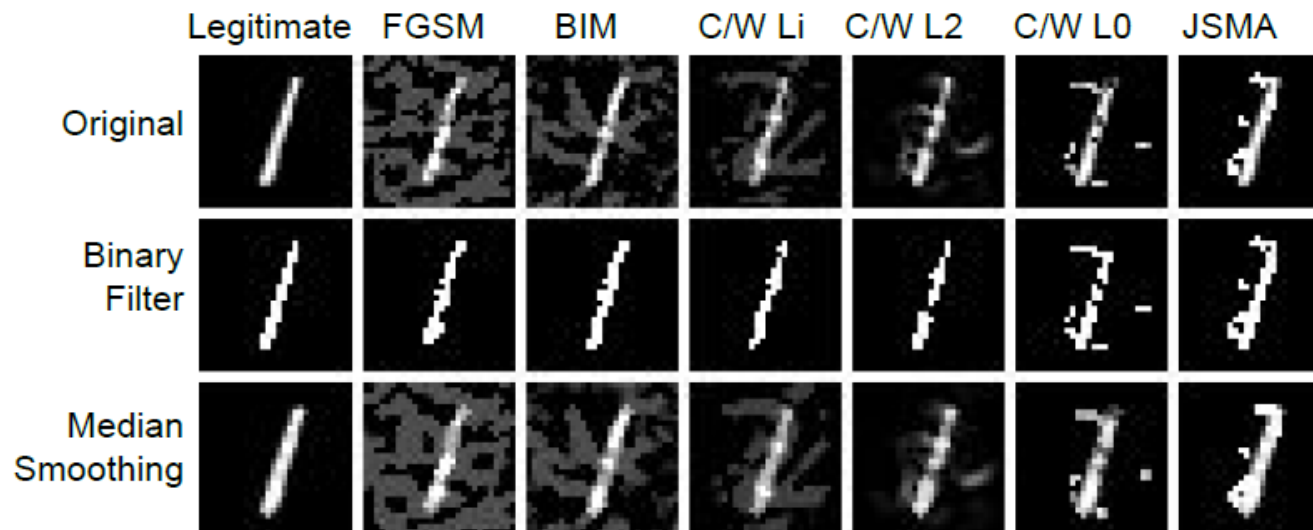
- [Hendrycks \(2016\) Early Methods for Detecting Adversarial Images](#)
- This work employs **PCA (principal component analysis)** to identify the most important principal components (PCs) of inputs (based on the ordering of the eigenvalues)
 - It was found that adversarial examples place **higher weights on the later PCs** that correspond to smaller eigenvalues, and clean images place uniform weights on all PCs
 - Thus, the distribution of PCs is used for detecting adversarial examples
 - E.g., the figures show the PCA coefficients for 3 clean (orange) and FGSM attacked images (blue) from CIFAR-10 dataset, for the largest 3,000 PCs
 - For all 3 adversarial images, the PCA coefficients are very small for the first 1,000 PCs, and they increase the most for the last 500 PCs (i.e., between 2,500 and 3,000 on the x-axis)



Prediction Consistency Methods

8.2 Adversarial Examples Detection: Prediction Consistency Methods

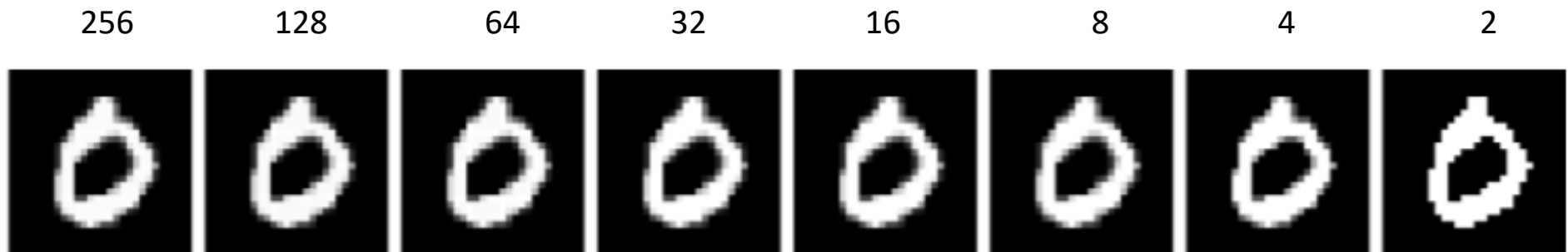
- [Xu \(2017\) Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks](#)
- *Feature squeezing defense* employs two methods for reducing the information in input features in images
 1. **Reducing the number of bits** for representing the pixels intensities
 2. Applying **spatial smoothing** to reduce the differences among the individual pixels
- E.g., the upper figures show a legitimate MNIST image and attacked examples
 - Second row: images with 1 bit intensity; Third row: spatially smoothed images



Feature Squeezing: Color Depth Reduction

8.2 Adversarial Examples Detection: Prediction Consistency Methods

- **Grayscale images** typically have $2^8 = 256$ intensity values for each pixel
 - Where a pixel with intensity of 0 is black, and intensity of 1 is a white pixel
- **Color images** have 3 channels – RGB (red, blue, and green)
 - The 3 channels encode $2^{24} = 16$ million different colors for the intensity of each pixel
- **Squeezing color bits** – is reducing the number of bits for each pixel in an image
 - E.g., for the grayscale image from MNIST, the figure shows variants with the number of bits reduced from $2^8 = 256$ intensities to $2^1 = 2$ intensities
 - The last image has only 2 bits (black and white pixels only, no gray pixels)



Feature Squeezing: Spatial Smoothing

8.2 Adversarial Examples Detection: Prediction Consistency Methods

- *Spatial smoothing* is a group of image-processing techniques for reducing image noise
 - Smoothing methods use the neighboring pixels to smooth each pixel
 - Common methods include: **median** smoothing, **mean** smoothing, and **Gaussian** smoothing
 - Removing noise from image can be used to reduce the impact of adversarial perturbations



Noisy image



Filtered by Mean Filter

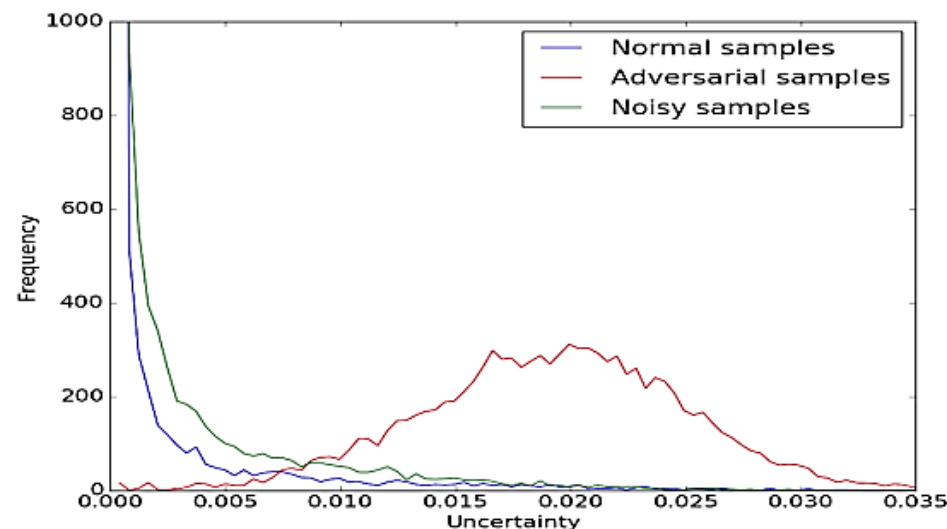


Filtered by Median Filter

Prediction Consistency Methods

8.2 Adversarial Examples Detection: Prediction Consistency Methods

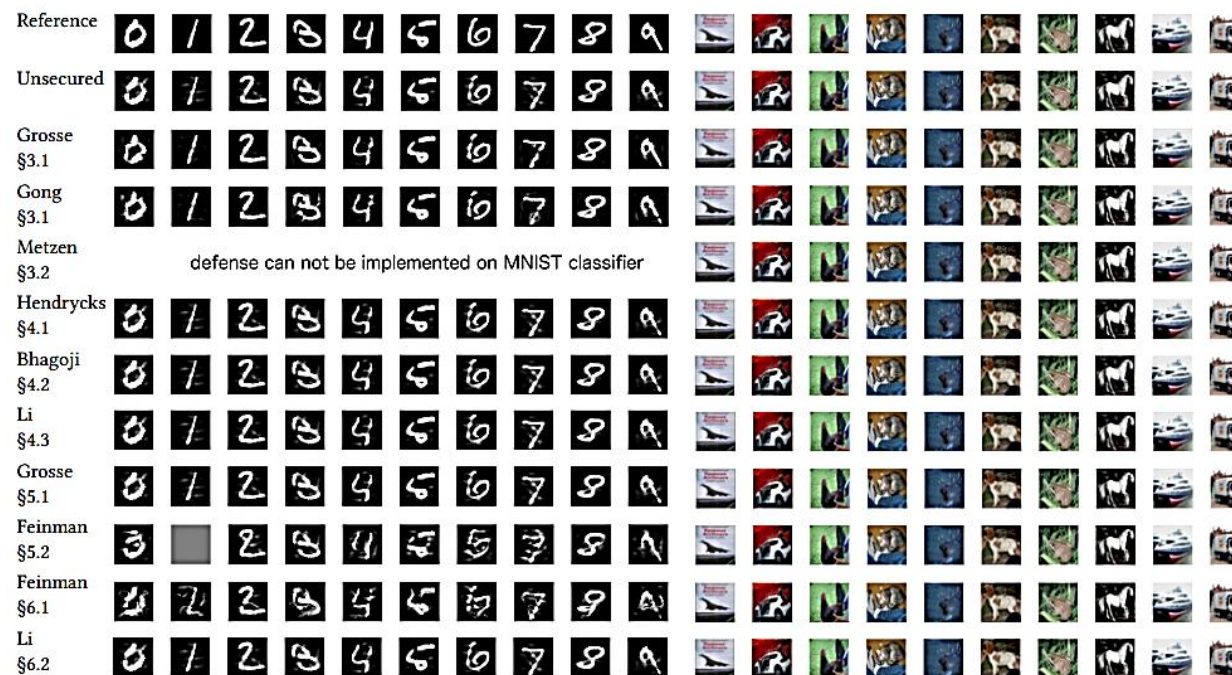
- [Feinman \(2017\) Detecting Adversarial Samples from Artifacts](#)
- The target classifier is **randomized using Dropout**
 - The dropout is set to eliminate 50% of the neurons in all layers
 - If the prediction by the randomized classifiers on sample x is significantly different than the prediction before the randomization, the sample x is probably adversarial
- Figure: the distribution of adversarial samples is notably different than the distribution of normal clean samples
 - The approach can detect adversarial samples with success rate of over 90% on MNIST and over 70% on CIFAR-10



Adversarial Examples Detection

8.2 Adversarial Examples Detection

- Carlini and Wagner published a paper in 2017 in which they showed that it is possible to defeat almost all Adversarial Examples Detection defense strategies
- [Carlini \(2017\) Adversarial Examples Are Not Easily Detected: Bypassing Ten Detection Methods](#)
 - In the paper, they analyzed 10 detection methods, and for each detector they explained how to bypass it, and make it inefficient for adversarial examples detection
 - Figure: adversarial images that **bypassed the 10 detector models**



Gradient Obfuscation

8.3 Gradient Obfuscation

- **Gradient obfuscation** defense methods deliberately hide the gradient of the model (from being used by an adversary)
 - Because most AML attacks are based on the model's gradient information, creating adversarial examples with such attacks becomes less successful
- These defense approaches can be grouped into:
 - **Exploding/vanishing gradients methods**, where the goal is to increase or reduce the gradient values in order to decrease the impact of adversarial examples
 - **Randomized gradients methods**, which apply some form of randomization of the DNN model as a defense strategy to confound the adversary
- Limitation of gradient obfuscation defenses is that they are designed to confound the adversaries, but they cannot eliminate the existence of adversarial examples



Defensive Distillation

8.3.1 Defensive Distillation

- [Papernot \(2016\) Distillation as a Defense to Adversarial Perturbations against Deep Neural Networks](#)
- *Defensive Distillation*
 - Applies **knowledge distillation** in NNs to defend against adversarial attacks
 - Obfuscates the gradients of NNs
- Results: reduced success rate of adversarial samples created by JSMA
 - MNIST: from 95.89% to 0.45%
 - CIFAR-10: from 87.89% to 5.11%

Defensive Distillation

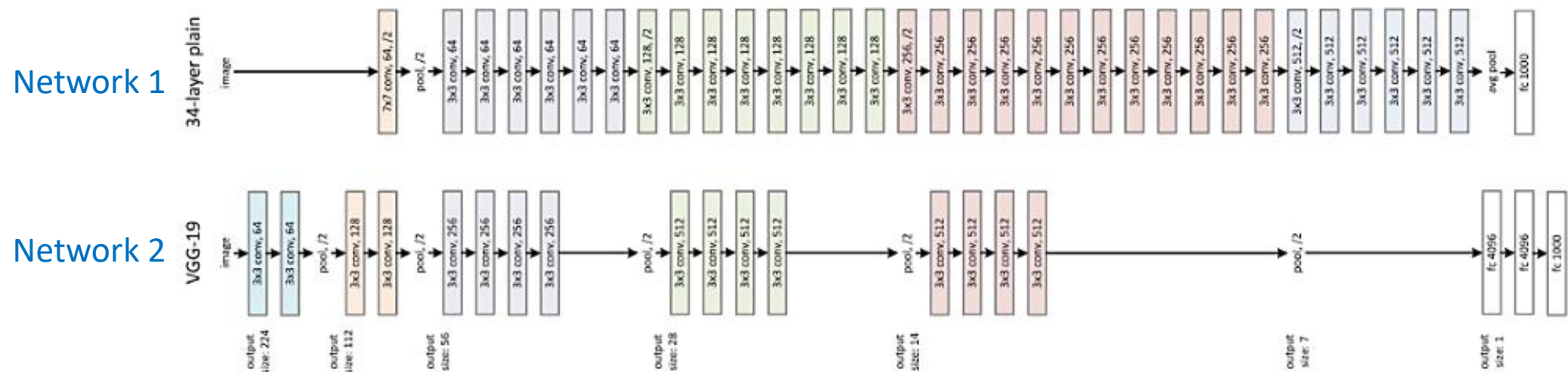
8.3.1 Defensive Distillation

- *Knowledge distillation* is the process of transferring knowledge from a **large NN** to a **smaller NN**
 - The goal is to achieve similar performance (accuracy) with the smaller NN model
- Motivation:
 - Knowledge distillation is motivated by **reducing the computational cost** for training or testing large NN models
 - E.g., use a smaller NN model for image classification on a resource-constrained device, such as a cell phone

Knowledge Distillation in NNs

8.3.1 Defensive Distillation

- In knowledge distillation, first a large network (*Network 1* below, or *teacher*) is trained
 - Then, the obtained output probability vectors produced by Network 1 are used as **soft labels** to train a smaller network (*Network 2*, or *student*)
- The aim is Network 2 to achieve **approximately the same accuracy** as Network 1, even though it has smaller capacity



Defensive Distillation

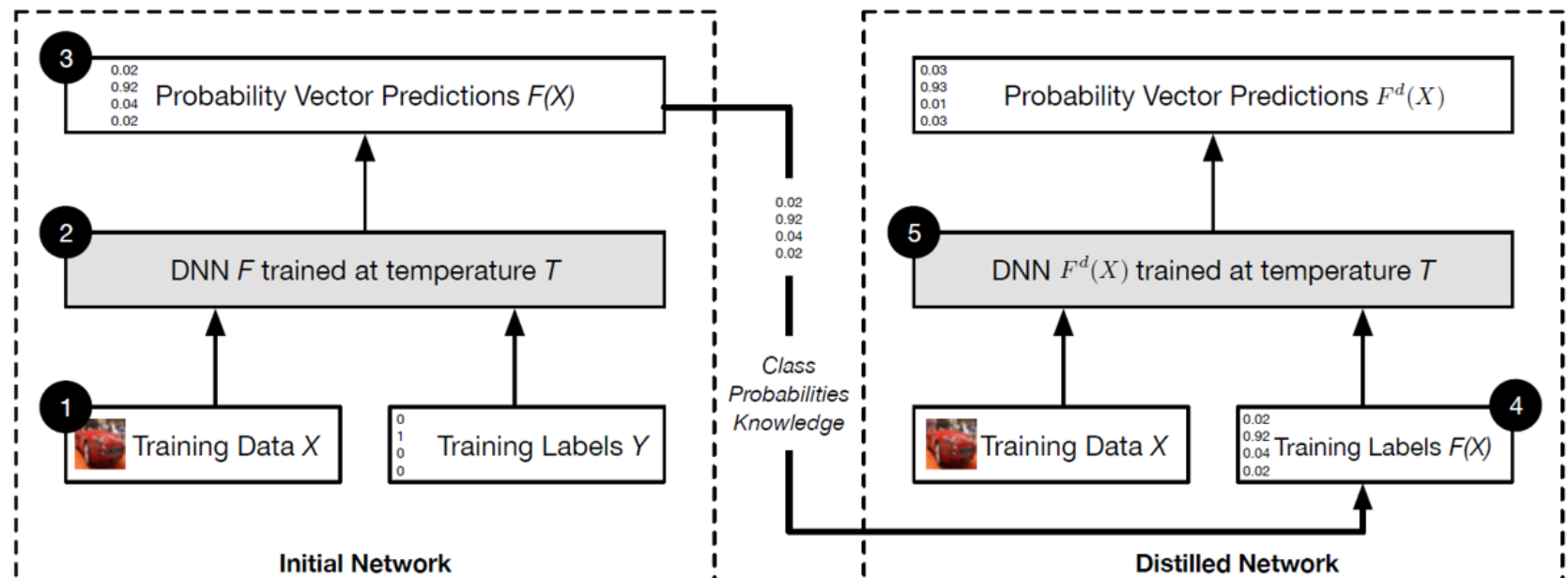
8.3.1 Defensive Distillation

- *Defensive distillation* applies the concept of knowledge distillation for defense training
- Approach:
 - Train Network 1
 - Use the predicted outputs by Network 1 as **soft labels** to train Network 2
 - **Soft labels**: have the output probability for each class (e.g., 0.1, 0.02, ..., 0.05)
 - In addition, to obtain the output probabilities, the logit values are divided by a constant T (called the **temperature**)
 - Train Network 2
 - Apply a constant temperature T again for the output probabilities
 - Network 1 and Network 2 have the same structure for distillation (Network 2 is not smaller)
 - Deploy Network 2
 - Set the temperature T for Network 2 to the initial value of 1
 - Deploy Network 2 for use by the end-users
- Motivation:
 - This defense approach causes small changes in the output class probabilities of Network 2 when the inputs are changed (e.g., by adding image pixel perturbations)
 - In other words, the gradients of Network 2 with respect to the inputs are small
 - This prevents adversaries from using gradient attacks to create adversarial examples

Defensive Distillation

8.3.1 Defensive Distillation

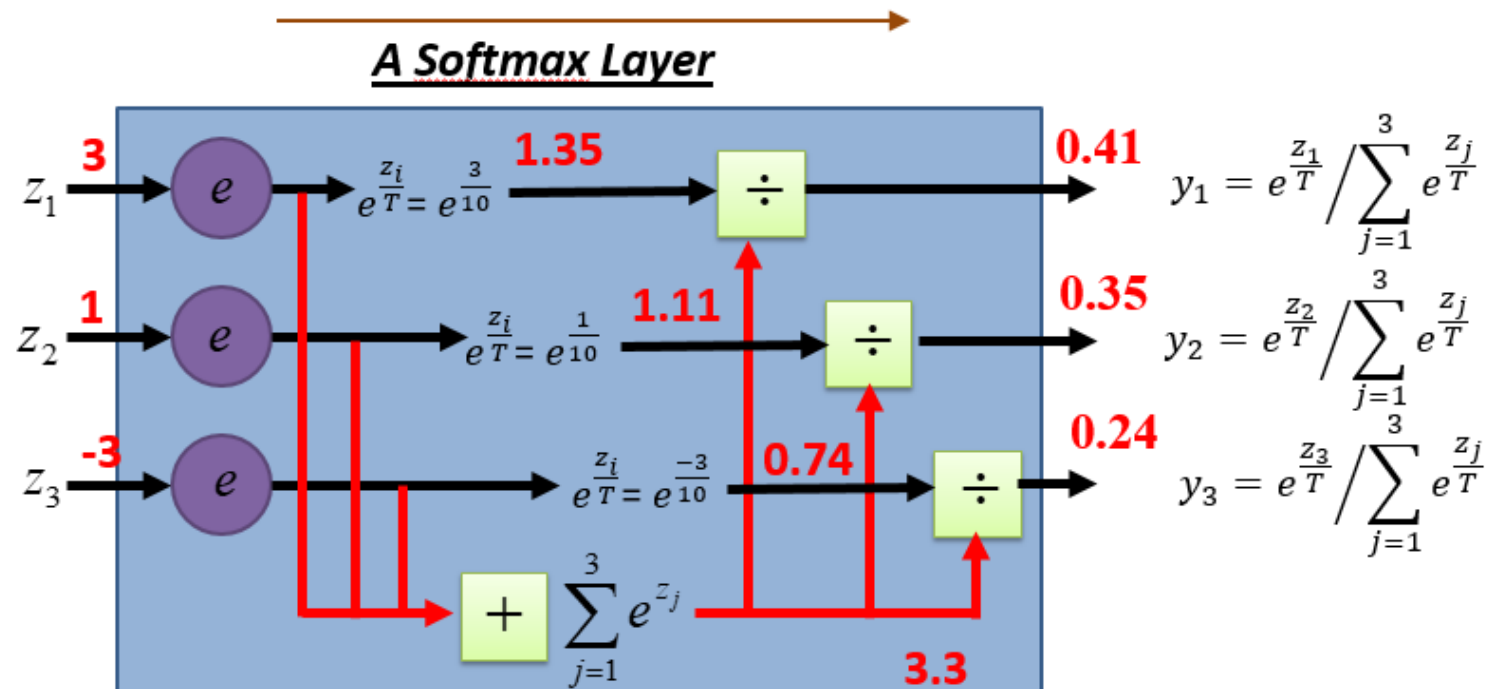
- Defense distillation uses the vector of class-probabilities $F(X)$ from the **Initial Network** (Network 1) as **soft labels** for training **Distilled Network** (Network 2)
 - This allows the Distilled Network to have additional knowledge about the training data X , regarding not only their true class label, but also about the probability of belonging to other classes
 - E.g., the soft label $[0.02, 0.92, 0.04, 0.02]$ in the figure is more informative than the hard label $[0, 1, 0, 0]$



Defensive Distillation

8.3.1 Defensive Distillation

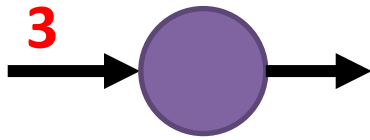
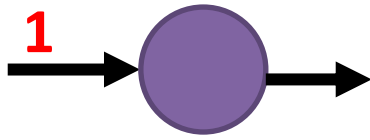
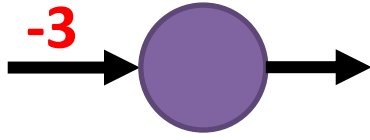
- For N -class classification, the softmax probability for class i is $F(X) = \left[\frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}} \right]$
- In defensive distillation, the softmax probability for class i is obtained by dividing the logits by a parameter T (the **temperature**), $F(X) = \left[\frac{e^{\frac{z_i}{T}}}{\sum_{j=1}^N e^{\frac{z_j}{T}}} \right]$



Defensive Distillation

8.3.1 Defensive Distillation

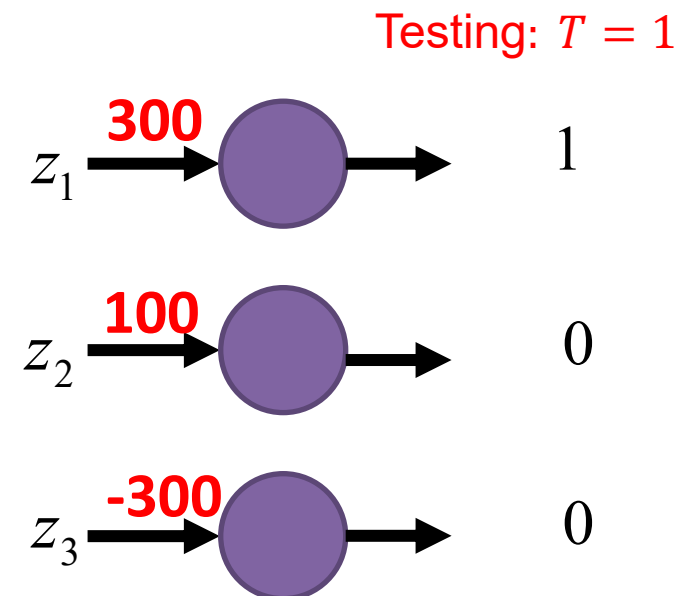
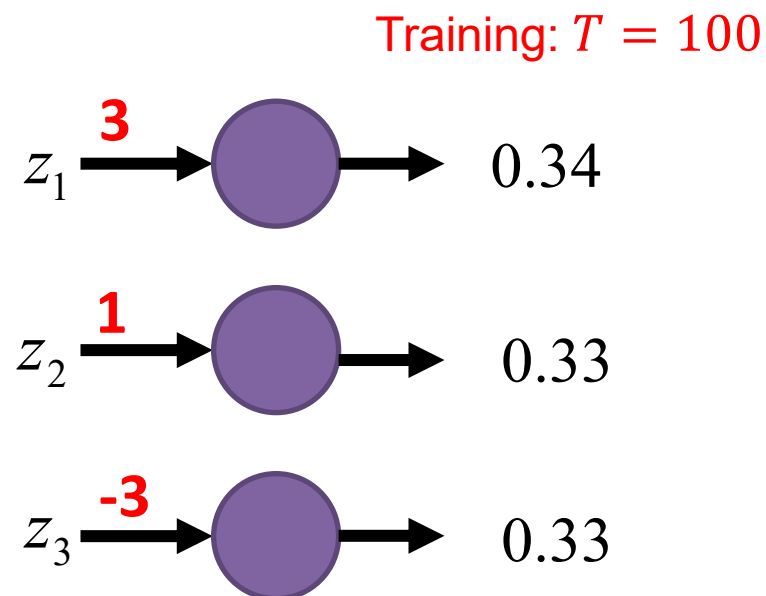
- The value of the temperature T is set to be greater than 1
 - Higher values of T result in large probabilities for each class
 - For $T \rightarrow \infty$, the probabilities for each class approach uniform distribution, i.e., they are $1/N$ for all classes

		$T = 1$	$T = 5$	$T = 10$	$T = 50$	$T = 100$
z_1		0.88	0.48	0.41	0.35	0.34
z_2		0.11	0.32	0.35	0.33	0.33
z_3		0.01	0.20	0.24	0.32	0.33

Defensive Distillation

8.3.1 Defensive Distillation

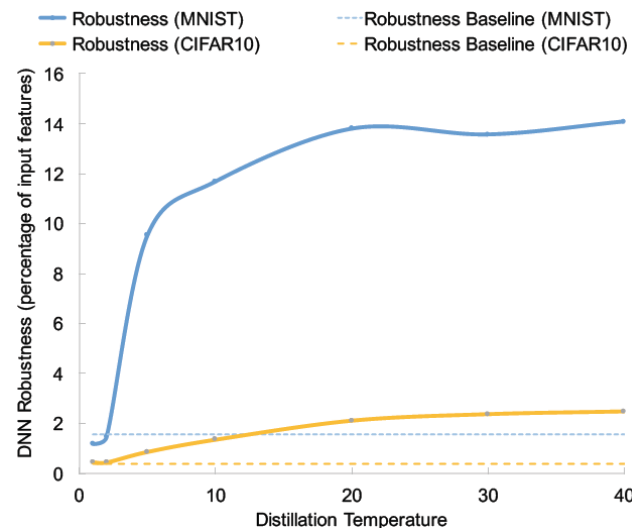
- Both the Initial Network $F(X)$ and the Distilled Network $F_d(X)$ are trained with a high softmax temperature
- At test time, the temperature is set back to $T = 1$
 - This causes the logits to be increased by a factor of T
 - The output probabilities will have a value close to 1 for the true class



Defensive Distillation

8.3.1 Defensive Distillation

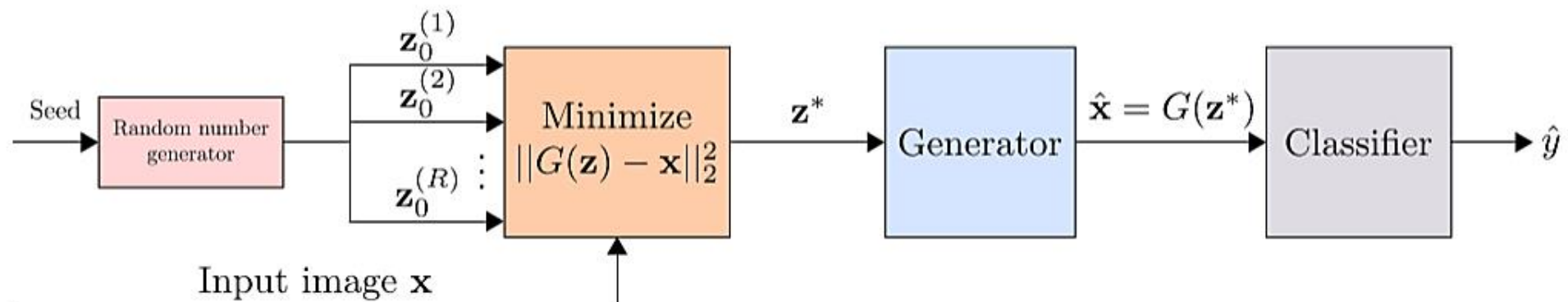
- The use of high temperature values improves the smoothness of the distilled model compared to the initial model
 - This makes the distilled model less sensitive to small changes in the inputs (e.g., applied via adversarial attacks)
 - Experiments show that distillation at high temperatures can **decrease the gradients** by factors up to 10^{30}
- Figure: Robustness for the DNN models trained on MNIST and CIFAR-10 versus the distillation temperature T
 - **Increased robustness** is observed for models trained on MNIST and CIFAR-10



Defense-GAN

8.3 Gradient Obfuscation

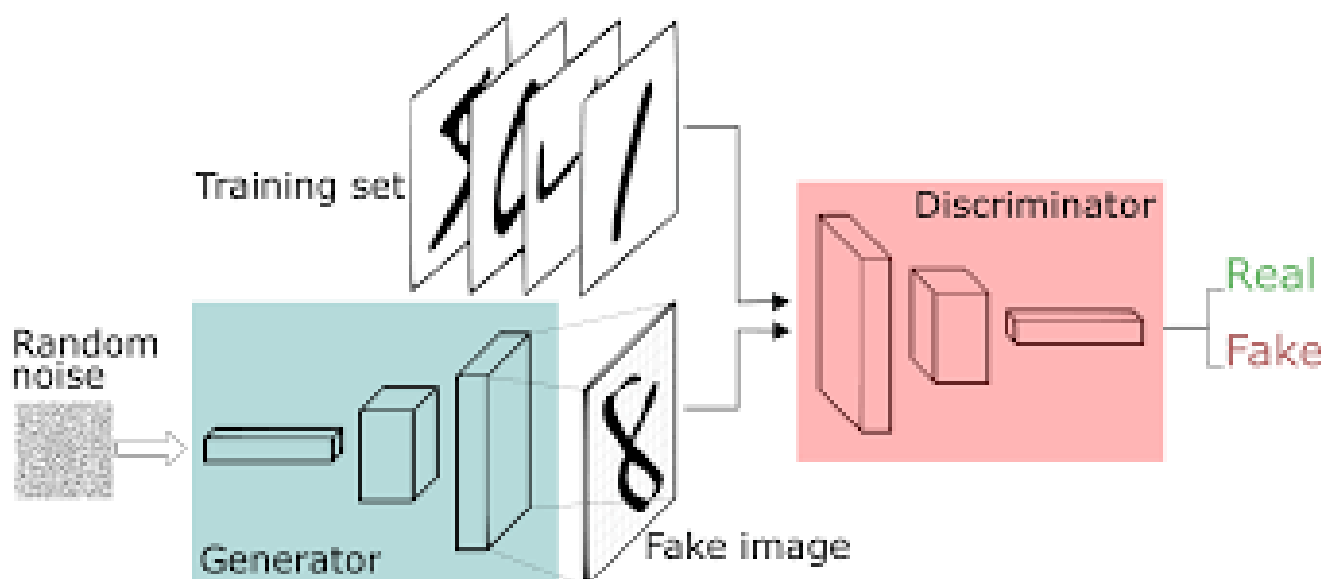
- [Samangouei \(2018\) Defense-GAN: Protecting Classifiers Against Adversarial Attacks Using Generative Models](#)
- *Defense-GAN* employs a **Generative Adversarial Network (GAN)** (see the next page) to purify adversarial examples prior to feeding them into a target classifier
 - GAN is trained to model the distribution of clean unperturbed images
 - The generator of GAN iteratively applies gradient descent to reconstruct input images and remove adversarial perturbations
- The combination of a generator and discriminator in Defense-GAN results in irregular small gradients (vanishing gradients)



GANs

8.3 Gradient Obfuscation

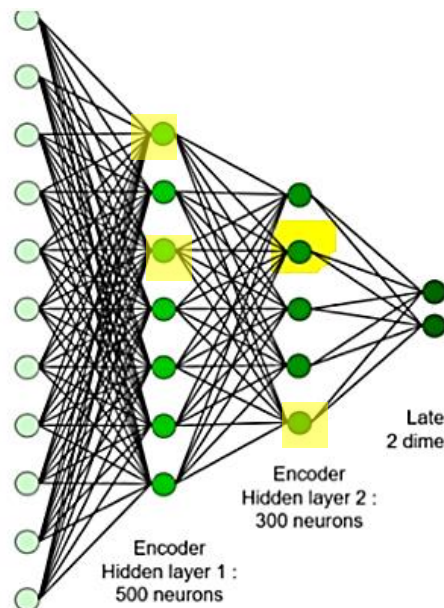
- *GANs – Generative Adversarial Networks*
 - **Generator** subnetwork – trained to generate new samples that are similar to the samples in the training set
 - **Discriminator** subnetwork – trained to discriminate real images from the training set and generated images by the generator
- The generator and discriminator are trained simultaneously where the generator improves in creating images that are similar to the real images, and the discriminator improves to distinguish real from fake images



Randomized Gradients

8.3 Gradient Obfuscation

- [Dhillon \(2018\) Stochastic Activation Pruning for Robust Adversarial Defense](#)
- *Stochastic Activation Pruning* removes a **subset of neurons' activations** in each layer
 - The remaining output activations in each layer are rescaled to normalize the inputs to the subsequent layer
 - This approach is similar to dropout layers, but for pruning it selects neurons with high activation values





Gradient Obfuscation Defenses

8.3 Gradient Obfuscation

- [Athalye, Carlini, Wagner \(2018\) - Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples](#)
- This work showed how to circumvent defenses based on gradient obfuscation
 - It was suggested that most of the defense strategies in this category give a false sense of security and have limitations
 - Existence of adversarial samples were demonstrated against those defenses

Robust Optimization

8.4 Robust Optimization

- *Robust optimization* methods aim to evaluate and improve the robustness of the target classifier to adversarial attacks
 - These approaches change the way model parameters are learned, in order to minimize the misclassification of adversarial examples
- These defense approaches can be categorized into three groups:
 - Adversarial training
 - Regularization methods
 - Certified defenses

Adversarial Training

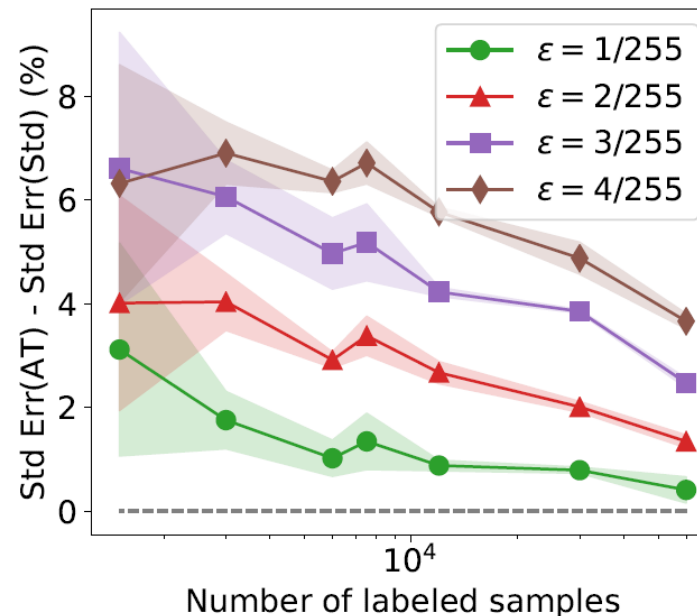
8.4.1 Adversarial Training

- *Adversarial training* is training or retraining the target classification model using adversarial examples
- The training dataset is augmented with adversarial examples produced by known types of attacks
 - E.g., for each clean example add one adversarial example to the training set
 - By adding adversarial examples x_{adv} with true label y to the training set, the model will learn that x_{adv} belongs to the class y
- Adversarial training is one of the most common adversarial defense methods currently used in practice
- Possible strategies:
 - Train the model from scratch using both regular and adversarial examples
 - Train the model on regular examples, and afterward fine-tune the model with adversarial examples

Adversarial Training

8.4.1 Adversarial Training

- Limitation of adversarial training is reduced accuracy on clean samples, known as *accuracy versus robustness trade-off*
 - The figure depicts the difference between the classification error of an adversarially trained model - Std Err (AT), and a model trained on clean examples only - Std Err (Std)
 - Note that adversarial training reduced the performance (between 3% and 7%)
 - Increasing the size of the dataset (number of labeled samples) reduces the gap
 - I.e., adversarially trained model with an infinitely large dataset would not suffer from reduced accuracy



Adversarial Training

8.4.1 Adversarial Training

- [Zhang \(2019\) Theoretically Principled Trade-off between Robustness and Accuracy](#)
- **TRADES** defense method addresses the trade-off between adversarial robustness and accuracy
 - TRADES stands for **TR**adeoff-inspired **A**dversarial **D**efense via **S**urrogate-loss minimization
- A robust classifier is trained by minimizing the following surrogate loss:

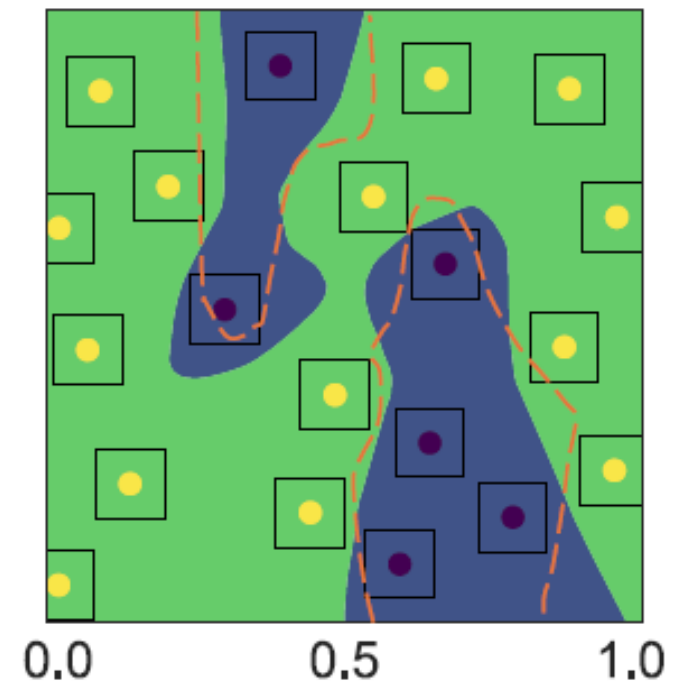
$$\min_x \mathbb{E} \left\{ \mathcal{L}(F(x), y) + \max_{\|x - x'\| \leq \epsilon} \mathcal{L}(F(x), F(x')) / \lambda \right\}$$

- The first term $\mathcal{L}(F(x), y)$ is employed to minimize the **standard (natural) error**, by ensuring that to **clean inputs** x the classifier F accurately assigns correct labels y
- The second term $\mathcal{L}(F(x), F(x'))$ minimizes the **robust error** to **adversarial samples**, by reducing the difference between the predictions of the classifier F on clean x samples, $F(x)$, and adversarial x' samples, $F(x')$
 - I.e., the label of a clean image $F(x)$ and the label of the corresponding adversarial sample $F(x')$ should be the same
 - PGD attack was used to find maximum perturbation $\|x - x'\|$ within a ball with radius ϵ
- The parameter λ balances the trade-off between the standard and robust error

Adversarial Training

8.4.1 Adversarial Training

- Figure: binary classification problem with yellow and black (clean) data points
 - Dashed orange line: decision boundary of a standard (natural) classifier trained only on clean samples
 - Blue area: decision boundary of a robust classifier trained with TRADES
- Both classifiers achieve 100% standard accuracy (correctly classify clean samples)
 - The blue classifier is more robust, because the second term in the loss function in TRADES $\mathcal{L}(F(x), F(x'))$ pushes the blue decision boundary away from the data points
 - I.e., the classifier will have higher accuracy in classifying samples in the neighborhood of the natural data points (that is, adversarial samples)



Adversarial Training

8.4.1 Adversarial Training

- Experimental evaluation on CIFAR-10
 - Increasing the ratio $1/\lambda$ decreases the **standard (natural) accuracy** on clean samples $\mathcal{A}_{\text{nat}}(f)$ from 91% to 82%, but increases the **robust accuracy** on adversarial samples $\mathcal{A}_{\text{rob}}(f)$ from 26% to 51%
 - The standard accuracy on clean images only is 95.2%

	CIFAR10	
$1/\lambda$	$\mathcal{A}_{\text{rob}}(f)$ (%)	$\mathcal{A}_{\text{nat}}(f)$ (%)
0.1	26.53 ± 1.1698	91.31 ± 0.0579
0.2	37.71 ± 0.6743	89.56 ± 0.2154
0.4	41.50 ± 0.3376	87.91 ± 0.2944
0.6	43.37 ± 0.2706	87.50 ± 0.1621
0.8	44.17 ± 0.2834	87.11 ± 0.2123
1.0	44.68 ± 0.3088	87.01 ± 0.2819
2.0	48.22 ± 0.0740	85.22 ± 0.0543
3.0	49.67 ± 0.3179	83.82 ± 0.4050
4.0	50.25 ± 0.1883	82.90 ± 0.2217
5.0	50.64 ± 0.3336	81.72 ± 0.0286

Adversarial Training

8.4.1 Adversarial Training

- [Raghunathan \(2020\) Understanding and Mitigating the Tradeoff Between Robustness and Accuracy](#)
- *RST (Robust Self-Training)* employs both labeled and unlabeled input examples to improve the robustness to adversarial examples
- Approach:
 - Train a classifier model F using pairs of clean samples x and ground-truth labels y
 - Evaluate the classifier F on unlabeled samples $\tilde{x}$, to obtain predictions $F(\tilde{x}) = \tilde{y}$ (referred to as pseudo-labels)
 - Create adversarial examples x' for the labeled samples, and $\tilde{x}'$ for unlabeled samples
 - Train a robust classifier G by minimizing a combined loss consisting of terms for the standard and robust accuracy on labeled and unlabeled samples:
$$\alpha \mathcal{L}_{\text{std-lab}}(F(x), y) + \beta \mathcal{L}_{\text{std-unlab}}(F(\tilde{x}), \tilde{y}) + \gamma \mathcal{L}_{\text{rob-lab}}(F(x'), y) + \lambda \mathcal{L}_{\text{rob-unlab}}(F(\tilde{x}'), \tilde{y})$$
- The addition of unlabeled samples improves the trade-off in comparison to other defense methods, i.e., increases the robust accuracy and also improves the standard accuracy

Adversarial Training

8.4.1 Adversarial Training

- Performance on CIFAR-10 against Wide ResNet28 model under ℓ_∞ perturbations of size $\epsilon = 8/255$
 - Comparison between supervised and semi-supervised (labeled and unlabeled data) methods are presented
 - PGD attack is used for generating adversarial samples
 - RST improved both the robust and standard accuracy
 - The authors used 50K labeled images and 500K unlabeled images
 - Two different versions of the robust loss $\mathcal{L}_{\text{rob-lab}}(F(x'), y)$ were implemented: PG-AT (projected gradient AT) and TRADES

Method	Robust Test Acc.	Standard Test Acc.	
Standard Training	0.8%	95.2%	} Vanilla Supervised
PG-AT (Madry et al., 2018)	45.8%	87.3%	
TRADES (Zhang et al., 2019)	55.4%	84.0%	
Standard Self-Training	0.3%	96.4%	} Semisupervised with same unlabeled data
Robust Consistency Training (Carmon et al., 2019)	56.5%	83.2%	
RST + PG-AT (this paper)	58.5%	91.8%	
RST + TRADES (this paper) (Carmon et al., 2019)	63.1%	89.7%	

Regularization Methods

8.4.2 Regularization Methods

- In ML, *regularization* is applied during training to control the complexity of the model
 - E.g., ℓ_2 or ℓ_1 **weight decay** regularization terms can be added to the loss function to penalize large values of the model parameters θ
 - Other *explicit* forms of regularization in NNs are: **dropout** and **early stopping**
 - *Implicit* regularization is also achieved by **batch normalization**
- Regularization prevents overfitting, and therefore improves *generalization*
- Several studies applied regularization to improve the robustness to adversarial examples
 - The motivation is that a regularized model with small magnitudes of the model parameters θ would produce smaller outputs to adversarially perturbed inputs

Certified Defenses

8.4.3 Certified Defenses

- *Certified defenses* methods verify the robustness of a trained model with respect to a metric/criterion
 - A certifiably robust classifier should have consistent predictions in a neighborhood around any input sample x (e.g., a neighborhood ball over ℓ_2 or ℓ_∞ norm)
- Considering the critical function of ML models in many applications, even if the model is deceived by one adversarial example can have important consequences
 - Designing ML models that are **certified** to be robust to adversarial perturbations under certain assumptions is a key direction for AML
 - For example, the work by Raghuathan produced a certificate for MNIST that guarantees that **no attack** with perturbation smaller than $\epsilon = 0.1 \approx 25/255$ can cause more than 35% test error
 - Even if the adversary has full access to the classifier and dataset, the certificate should hold under the provided conditions

Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

- [Cohen \(2019\) Certified Adversarial Robustness via Randomized Smoothing](#)
- This work employs *Randomized Smoothing* to design a classifier that is certifiably robust to adversarial perturbations under the ℓ_2 norm
 - It introduced the first certificate on ImageNet
 - E.g., the certificate guarantees top-1 accuracy with ResNet50 on ImageNet of 49% under adversarial perturbations with norm $\ell_2 < 0.5$
 - Previous certified defenses were designed for smaller datasets, and were difficult to scale to larger NN models that are commonly used for ImageNet
- The main idea is to apply Gaussian noise to input images, and use the most probable class on the perturbed images as a robust prediction
 - Robustness is achieved by overpowering small adversarial perturbations with large random Gaussian perturbations
- Advantages:
 - A simple approach that can be applied to large NNs
 - It does not make any assumptions about the used target classifier

Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

- Assume we have a **target classifier** f that maps inputs x to labels y , i.e., $f(x) = y$
- The approach creates corrupted versions of the image x by applying isotropic **Gaussian noise** with 0 mean and variance σ^2 , i.e., $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$
 - Left figure: input sample x ; Right figure: image corrupted with Gaussian noise $x + \varepsilon$
- A **smoothed classifier** g is obtained by adopting the **most probable class** by the classifier f on many noise-corrupted images $x + \varepsilon$
 - The added random noise improves the robustness to adversarial perturbations



Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

- To design a **smoothed classifier** g at the input sample x requires to identify the most likely class $\hat{c}_A$ returned by the target classifier f on noisy images
 - Step 1: create n versions of x corrupted with Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$
 - Step 2: evaluate the predictions by target classifier for all corrupted images, $f(x + \varepsilon)$
 - Step 3: identify the top two classes $\hat{c}_A$ and $\hat{c}_B$ with the highest number of predictions on $f(x + \varepsilon)$
 - Step 4: if n_A (number of predictions by f for the top class $\hat{c}_A$) is much greater than n_B (number of predictions for the second highest class $\hat{c}_B$), return $\hat{c}_A$ as the prediction by $g(x)$
 - Otherwise, if $n_A - n_B < \alpha$, abstain from making a prediction by $g(x)$

evaluate g at x

function PREDICT(f, σ, x, n, α)

counts $\leftarrow$ SAMPLEUNDERNOISE(f, x, n, σ)

$\hat{c}_A, \hat{c}_B \leftarrow$ top two indices in counts

$n_A, n_B \leftarrow$ counts[$\hat{c}_A$], counts[$\hat{c}_B$]

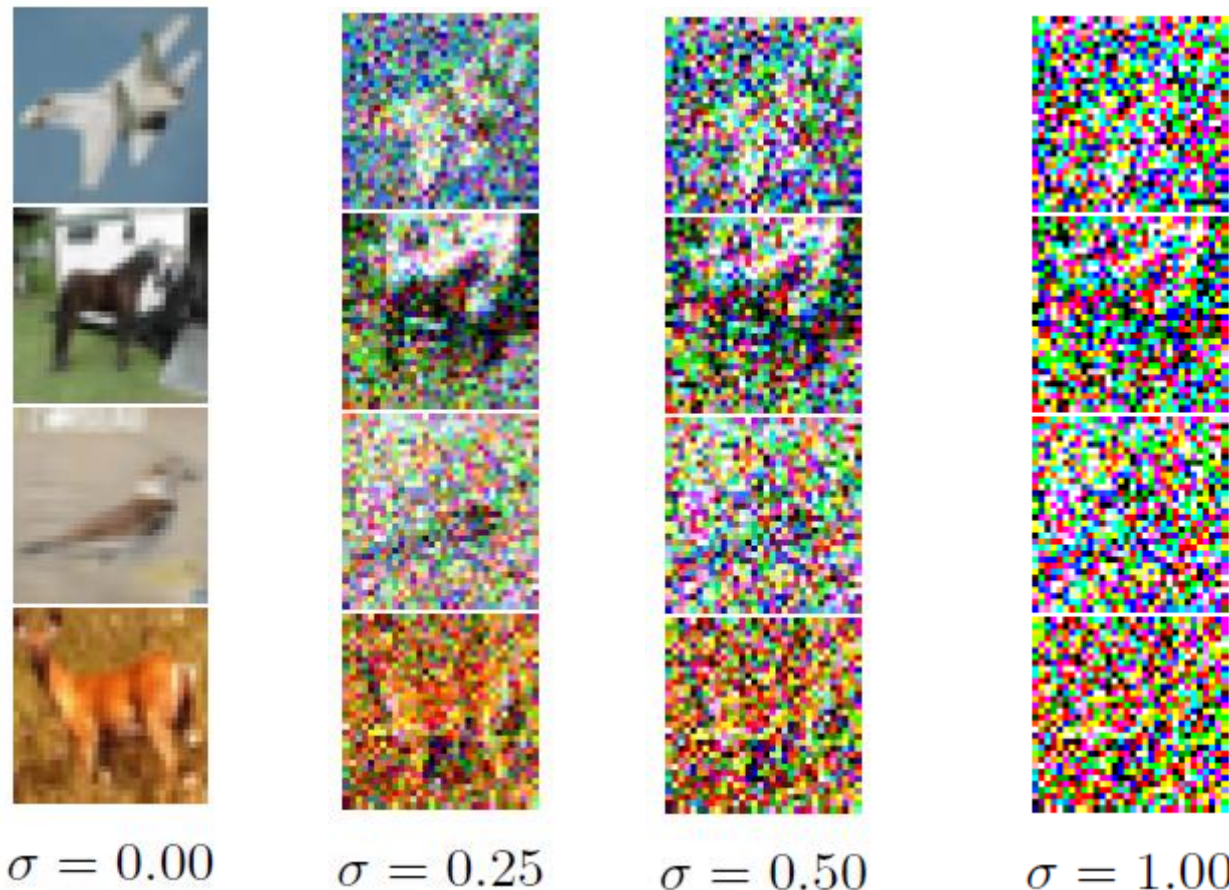
if BINOMPVALUE($n_A, n_A + n_B, 0.5$) $\leq \alpha$ **return** $\hat{c}_A$

else return ABSTAIN

Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

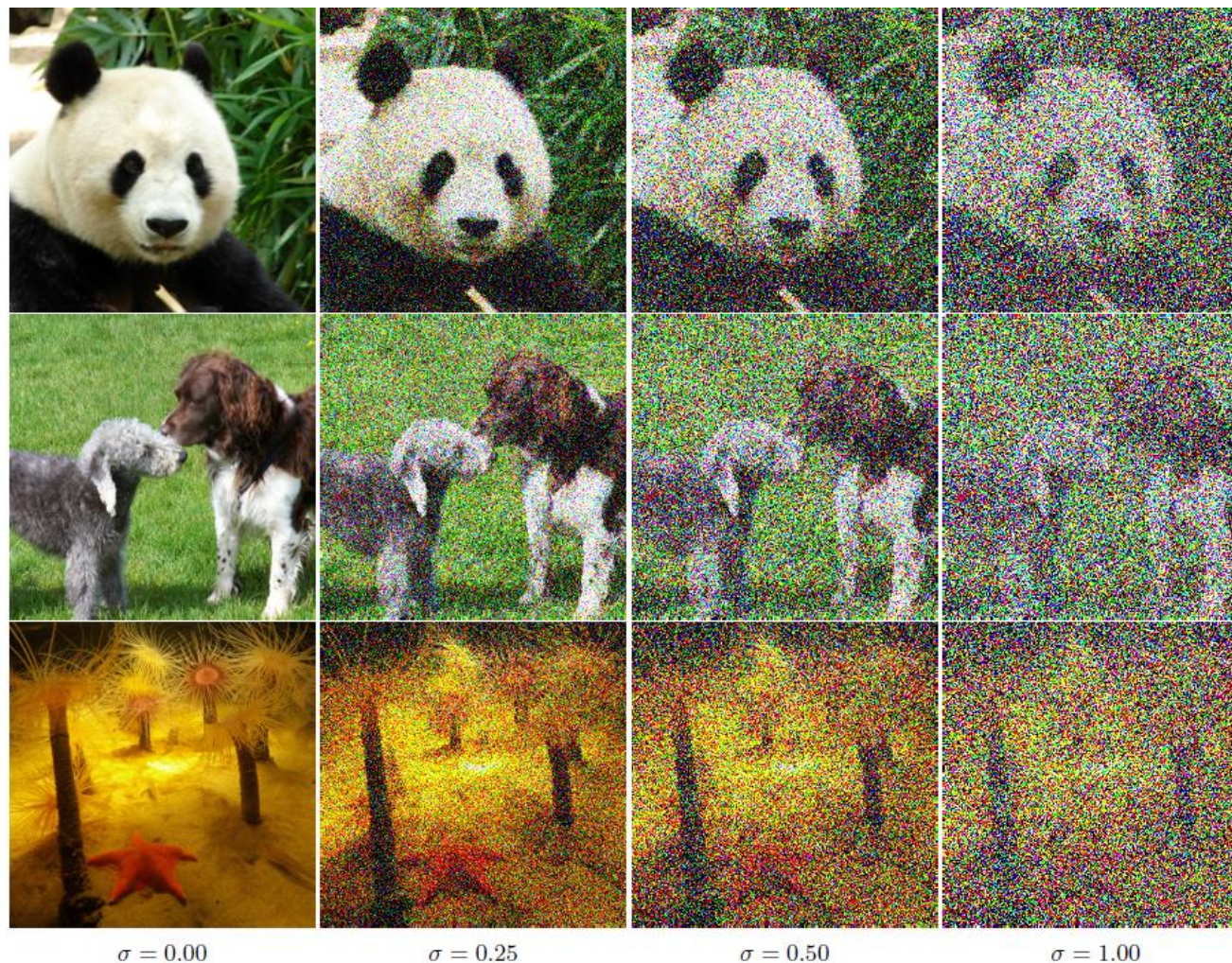
- Examples of noisy images from CIFAR-10 with varying the level of Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ from $\sigma = 0$ to $\sigma = 1$
 - Pixel values greater than 1 (i.e., 255/255) or less than 0 were clipped to 1 or 0



Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

- Examples of noisy images from ImageNet with varying the level of Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ from $\sigma = 0$ to $\sigma = 1$



Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

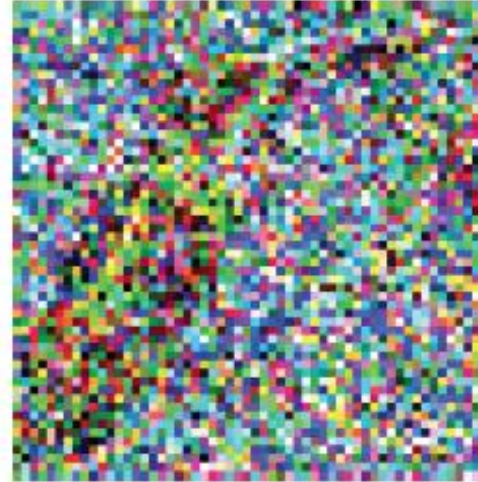
- Images with higher resolution can tolerate Gaussian noise $\mathcal{N}(0, \sigma^2 I)$ with higher levels of σ
 - Therefore, smoothing can be performed with a larger σ in high resolution images
 - The noisy high-resolution image preserved the class-defining features better



Clean 56×56 image



Clean 224×224 image



Noisy 56×56 image
($\sigma = 0.5$)



Noisy 224 ×224 image
($\sigma = 0.5$)

Randomized Smoothing Certificate

8.4.3.1 Randomized Smoothing Certificate

- The table shows the certified top-1 accuracy by ResNet50 on ImageNet with the random smoothing approach
 - Top row: the certificate guarantees top-1 accuracy of 49% under adversarial perturbations $\ell_2 < 0.5$
 - This is achieved with noise level of $\sigma = 0.25$
 - For any perturbation with radius $\ell_2 < 0.5$, the robust classifier will correctly predict the class
 - Note that perturbation with ℓ_2 norm < 0.5 is fairly small
 - For example, perturbation with $\ell_2 = 1$ can change one pixel by 1 ($=255/255$), or change 10 pixels by 0.3 ($\approx 80/255$), or change 1,000 pixels by 0.03 ($\approx 8/255$)
 - Increasing the ℓ_2 radius from 0.5 to 3.0 reduces the certified accuracy
 - For comparison, the standard top-1 accuracy on unperturbed images by the smoothed classifier is 67%

ℓ_2 RADIUS	BEST σ	CERT. ACC (%)	STD. ACC(%)
0.5	0.25	49	67
1.0	0.50	37	57
2.0	0.50	19	57
3.0	1.00	12	44

Certified Adversarial Robustness for Free

8.4.3.2 Certified Adversarial Robustness for Free

- [Carlini \(2022\) \(Certified!!\) Adversarial Robustness for Free!](#)
- This paper focuses on achieving certified adversarial robustness against ℓ_2 norm bounded perturbations
- Methodology
 - Rely exclusively on off-the-shelf pretrained models
 - Employ the Randomized Smoothing certificate approach
 - Combine with a pretrained denoising diffusion probabilistic model (DDPM)
- Advantages
 - Method simplicity: achieved using pretrained diffusion models and image classifiers
 - Diffusion models are already trained to denoise Gaussian noise and there is no need for fine-tuning or re-training of model parameters (that is why the paper title uses the term “for free”)
- Results
 - Certified 71% accuracy on ImageNet under adversarial perturbations, constrained within ℓ_2 norm of $R = 0.5$

Randomized Smoothing

8.4.3.2 Certified Adversarial Robustness for Free

- **Randomized smoothing** is the same approach we covered in the randomized smoothing certificate, based on the paper by Cohen (2019)
 - To compute a lower-bound on the robust radius R of a classifier f , for an image x and perturbation δ , estimate probabilities $P[f(x + \sigma) = c]$ for each class label c by sampling a large number of perturbations σ
- For a comparison, let's outline the steps in this approach again
- Steps:
 1. Add Gaussian noise σ to input x
 2. Use a classifier model to predict the class
 3. Repeat N times, take majority vote over the predicted labels
 4. If the top class wins by sufficient margin, certify robustness radius R under ℓ_2 norm

Denoised Smoothing

8.4.3.2 Certified Adversarial Robustness for Free

- *Denoised smoothing* approach proposed by Salman (2020) is an extension of randomized smoothing method
 - It combines a **standard classifier** with a **denoiser model**
- Steps:
 1. Add Gaussian noise σ to input x
 2. Denoise the input x using a *custom-trained denoiser model*
 3. Use a *standard classifier* to predict the class of the denoised inputs
 4. Repeat N times, take majority vote over the predicted labels
 5. If the top class wins by sufficient margin, certify robustness radius R under ℓ_2 norm
- The main difference of Denoised Smoothing compared to the Randomized Smoothing is inserting a denoising step (Step 2), which enables using a standard classifier
- The denoiser model is trained using Gaussian noise augmentation
 - It improves the accuracy of the standard classifier on noisy inputs, widening the certified robustness radius R

Certified Adversarial Robustness for Free

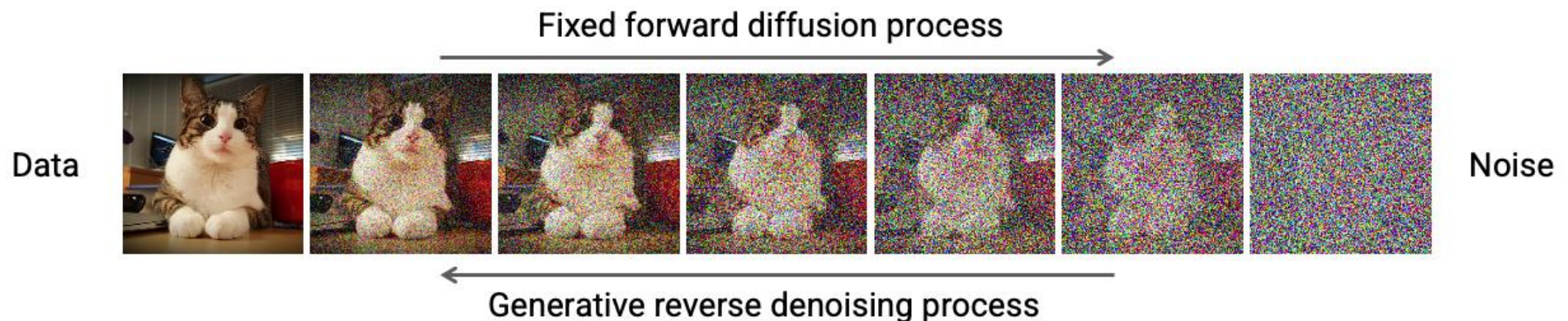
8.4.3.2 Certified Adversarial Robustness for Free

- The proposed approach **Certified Adversarial Robustness for Free** is similar to the Denoised Smoothing approach, but instead of using a custom-trained denoiser model in Step 2, it uses a **denoising diffusion probabilistic model (DDPM)**
- Steps:
 1. Add Gaussian noise σ to input x
 2. Denoise the input x using a *pretrained DDPM*
 3. Use a *standard classifier* to predict the class of the denoised inputs
 4. Repeat N times, take majority vote over the predicted labels
 5. If the top class wins by sufficient margin, certify robustness radius R under ℓ_2 norm
- DDPM are already trained to denoise Gaussian noise, therefore, no additional training is needed
- This approach achieves stronger certified robustness with zero extra training

Denoising Diffusion Probabilistic Models

8.4.3.2 Certified Adversarial Robustness for Free

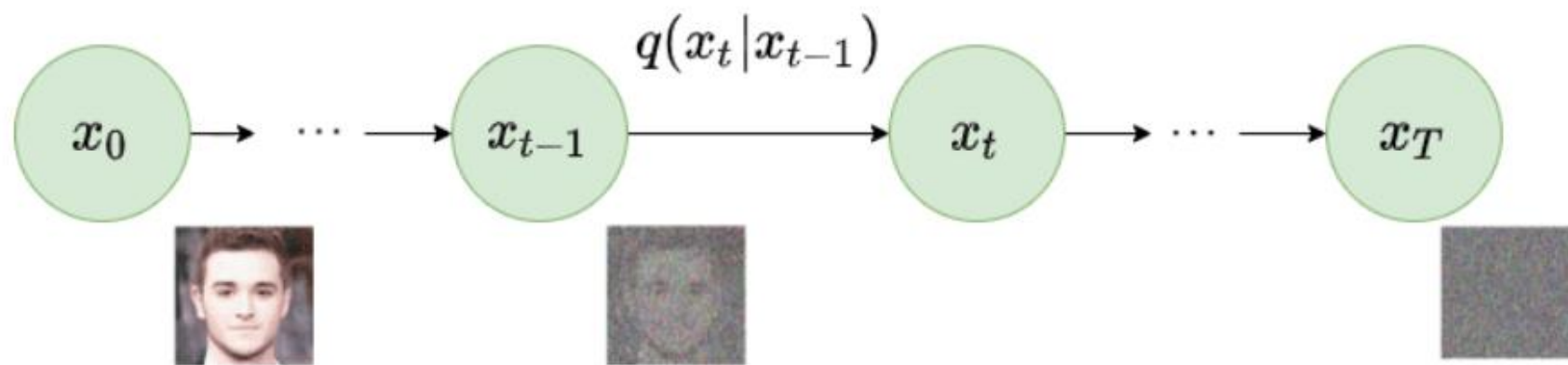
- *Denoising diffusion probabilistic models (DDPM)* are generative models that learn to reverse a diffusion noise process
 - Approach: add random noise to images, and train NN to learn to iteratively denoise the images
- Most current models for text-to-image or text-to-video generation (e.g., Google Imagen, Midjourney, OpenAI Sora) employ DDPMs
 - These models integrate LLMs (for text understanding) with diffusion models (for image/video generation)



Denoising Diffusion Probabilistic Models

8.4.3.2 Certified Adversarial Robustness for Free

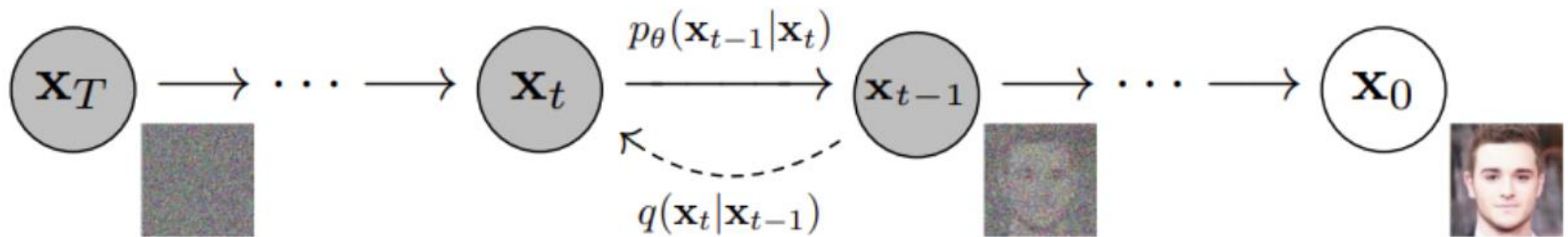
- **Forward diffusion process (fixed)**
 - Starting with a clean image from target data distribution, add random Gaussian noise over many steps to convert the image to pure noise
 - The added Gaussian noise at each step has variance that is either constant over the entire process or is gradually changed (e.g., using cosine or tanh function)
 - This process is fixed, and the obtained images at each step are used in the next step (reverse diffusion process) to train a NN



Denoising Diffusion Probabilistic Models

8.4.3.2 Certified Adversarial Robustness for Free

- **Reverse denoising process (learned)**
 - Starting with pure noise, train a NN to recover a realistic image
 - At each step, the NN learns the amount of noise that needs to remove in order to recover a clean denoised image
 - I.e., at step t , the NN predicts how much noise to remove to obtain a slightly denoised image at step $t - 1$



Results

8.4.3.2 Certified Adversarial Robustness for Free

- ImageNet certified top-1 accuracy for prior defenses based for radius R having values 0.5, 1.0, 1.5, 2.0 and 3.0
 - Accuracies on clean images are shown in parentheses
 - Achieved 71.1 % robust accuracy and 82.2% clean accuracy for $R = 0.5$
- Configuration used by the authors:
 - Pretrained 552M parameter diffusion model used as denoiser model
 - BEiT 305M parameter vision transformer used as image classifier

Method	Off-the-shelf	Extra data	Certified Accuracy at R level				
			0.5	1.0	1.5	2.0	3.0
PixelDP (Lecuyer et al., 2019)	○	✗	(33.0) 16.0	-	-	-	-
RS (Cohen et al., 2019)	○	✗	(67.0) 49.0	(57.0) 37.0	(57.0) 29.0	(44.0) 19.0	(44.0) 12.0
SmoothAdv (Salman et al., 2019)	○	✗	(65.0) 56.0	(54.0) 43.0	(54.0) 37.0	(40.0) 27.0	(40.0) 20.0
Consistency (Jeong & Shin, 2020)	○	✗	(55.0) 50.0	(55.0) 44.0	(55.0) 34.0	(41.0) 24.0	(41.0) 17.0
MACER (Zhai et al., 2020)	○	✗	(68.0) 57.0	(64.0) 43.0	(64.0) 31.0	(48.0) 25.0	(48.0) 14.0
Boosting (Horváth et al., 2022a)	○	✗	(65.6) 57.0	(57.0) 44.6	(57.0) 38.4	(44.6) 28.6	(38.6) 21.2
DRT (Yang et al., 2021)	○	✗	(52.2) 46.8	(55.2) 44.4	(49.8) 39.8	(49.8) 30.4	(49.8) 23.4
SmoothMix (Jeong et al., 2021)	○	✗	(55.0) 50.0	(55.0) 43.0	(55.0) 38.0	(40.0) 26.0	(40.0) 20.0
ACES (Horváth et al., 2022b)	◐	✗	(63.8) 54.0	(57.2) 42.2	(55.6) 35.6	(39.8) 25.6	(44.0) 19.8
Denoised (Salman et al., 2020)	◑	✗	(60.0) 33.0	(38.0) 14.0	(38.0) 6.0	-	-
Lee (Lee, 2021)	●	✗	41.0	24.0	11.0	-	-
Ours	●	✓	(82.8) 71.1	(77.1) 54.3	(77.1) 38.1	(60.0) 29.5	(60.0) 13.1